

Trabajo Práctico Integrador

El objetivo de este trabajo es desarrollar un **programa completo en Java (modo consola)** que modele un **escenario realista elegido por los estudiantes**, aplicando los conceptos vistos.

El proyecto debe permitir **el ingreso y la gestión de datos** (por ejemplo, clientes, productos, alumnos, empleados, vehículos, etc.), aplicando **programación estructurada, POO, colecciones y buenas prácticas de desarrollo colaborativo con Git**.

Los grupos se deben inscribir en <https://forms.gle/pJz2wxxSGoYjMfDm9>

Cada grupo deberá **proponer su propio escenario o problema real** a modelar, el cual debe cumplir las siguientes condiciones:

1. **Debe involucrar objetos o entidades** (por ejemplo: *Producto, Alumno, Empleado, Libro, Pedido, Cliente, etc.*).
2. El sistema debe permitir:
 - **Agregar, buscar, modificar, eliminar y listar** registros.
 - Al menos **una operación adicional** según el contexto (por ejemplo: calcular promedio, aplicar descuentos, registrar asistencia, procesar ventas, etc.).
3. Todo el programa debe funcionar **únicamente por consola** (sin interfaz gráfica).
4. Debe ser **original, coherente con la realidad y posible de implementar** en las semanas de cursado.

Ejemplos de posibles escenarios (a modo orientativo):

- Sistema de gestión de **biblioteca**.
- Registro de **alumnos y calificaciones**.
- Control de **inventario de productos**.
- Agenda de **contactos personales**.
- Administración de **vehículos y mantenimientos**.

- Gestión de **reservas en un restaurante o gimnasio**.

Trabajo en equipo y Git

El trabajo será grupal (2 a 4 integrantes).

Cada integrante deberá:

- Crear su **rama personal** (por ejemplo, `rama-lucas`, `rama-ana`, `rama-martin`).
- Realizar **commits frecuentes y descriptivos**.
- Desarrollar al menos **una clase o funcionalidad principal**.
- Integrar su trabajo a la rama principal (`main` o `develop`) mediante **merge** y resolver conflictos si los hubiera.
- Documentar en el repositorio su participación y los aportes de cada uno.

Entrega:

- Enviar al profesor (facundouferer@gmail.com) el enlace del repositorio Git (GitHub o GitLab).
- Exponer el trabajo explicando el **diseño, estructura y lógica** del programa.

Requisitos de desarrollo

Programación estructurada

El programa debe incluir:

1. **Ingreso de datos** por teclado con `Scanner`.
2. Estructuras de control:
 - Secuenciales, condicionales (`if`, `else if`, `switch`) y repetitivas (`for`, `while`, `do-while`, `for-each`).
 - Uso de `break` y `continue` donde sea necesario.
3. Uso de **arrays o matrices** en alguna parte del código.
4. Manejo de **Strings** (concatenación, comparación, búsqueda, etc.).
5. Creación de **subprogramas** (métodos) con **parámetros** y distintos **alcances de variables**.
6. Ejemplo de **recursividad simple** (por ejemplo, un menú que se repite hasta salir).

Paradigma Orientado a Objetos

Debe implementarse al menos:

1. **Dos o más clases** que representen entidades del problema (por ejemplo, **Producto**, **Cliente**, **Pedido**).
2. Uso de:
 - **Constructores** (por defecto y parametrizados).
 - **Getters y Setters**.
 - **Modificadores de acceso** (**public**, **private**, **protected**).
 - **Métodos estáticos** o de utilidad.
3. Aplicar los **pilares de la POO**:
 - **Encapsulamiento**, **herencia** y **polimorfismo** (por ejemplo, una clase hija que sobrescriba un método).
4. Uso de **interfaces** (al menos una), e implementación de **una interfaz funcional** con **expresión lambda**.
5. Uso de **métodos de referencia** y colecciones con **forEach**, **anyMatch**, **map** o **removeIf**.
6. Uso de **Wrappers** (**Integer**, **Double**, etc.) y **autoboxing/unboxing**.

TAD y Colecciones

Debe incluir:

1. Uso de al menos **una colección del Java Collections Framework**, como:
 - **ArrayList**, **LinkedList**, **HashSet**, **TreeSet**, **HashMap** o **Queue**.
2. Implementar al menos una **estructura propia** simple (por ejemplo, una lista enlazada para registro de operaciones).
3. Uso de:
 - **Iteradores**, **comparadores** (**Comparator** o **Comparable**).
 - **equals()** y **hashCode()** cuando sea necesario.
4. Métodos funcionales de colección (**removeIf**, **anyMatch**, **allMatch**, **forEach**).
5. **Ordenamiento** y **filtrado** de datos.

Manejo de excepciones

- **try-catch-finally** para controlar errores de entrada o división por cero.
- Excepción personalizada, como **ElementoNoEncontradoException**.

Evaluación

Criterio	Ponderación
Cumplimiento de requisitos técnicos	30%

Aplicación correcta de los conceptos	25%
Diseño y uso correcto de POO y colecciones	20%
Trabajo colaborativo y manejo de Git	35%

Ejemplo general de interacción esperada

Cada grupo adaptará el flujo según su tema elegido, pero el comportamiento base debe ser similar:

```
===== SISTEMA DE GESTIÓN DE BIBLIOTECA =====
```

1. Agregar libro
2. Eliminar libro
3. Buscar libro por título
4. Listar todos los libros
5. Registrar préstamo
6. Mostrar libros disponibles
7. Salir

```
Seleccione una opción: 1
```

```
Ingrese código: 101
```

```
Ingrese título: "Java para principiantes"
```

```
Ingrese autor: "Juan Pérez"
```

```
Ingrese stock: 5
```

```
Libro agregado correctamente.
```